TP3 — CPU Scheduling Profiler

Laboratório de Sistemas Operacionais — PUCRS 2026/2 · Pedro Jardim Machado e Lucas Lorenzi da Silva · 10/11/2026

1. Objetivo e ambiente

Avaliar FIFO, RR, IDLE e LOW_IDLE no Linux 4.13.9 (Buildroot/i386) em QEMU. Os testes principais usaram 1 CPU, buffer de 5.000.000 posições, quatro threads (duas no misto: A=LOW_IDLE, B=IDLE) e traces sched_switch analisados no KernelShark. O kernel usa WEIGHT_LOW_IDLEPRIO=1 e WMULT_LOW_IDLEPRIO=4294967295.

2. Implementação e medida

Cada thread grava sua letra na próxima posição do buffer; um semáforo binário protege o índice. As políticas são aplicadas antes da largada; FIFO/RR usam prioridade 10. O programa compacta letras consecutivas e imprime escalonamentos estimados (trocas de contexto + primeira entrada), caracteres escritos e CPU por thread.

$ ./sched_profiler 5000000 4 SCHED_RR

Pos-processado: ABCDABCDABCDABCD

A: 10 escalonamentos, 1.206.817 caracteres, 20,98% CPU

B: 9 escalonamentos, 1.777.829 caracteres, 31,39% CPU

C: 9 escalonamentos, 1.234.617 caracteres, 26,47% CPU

D: 8 escalonamentos, 780.737 caracteres, 21,17% CPU

3. Resultados (uma execução por caso)

A ordem nas células é A/B/C/D; no teste misto, A/B. O tempo entre parênteses é CPU total das threads, em segundos.

Política (CPU s)	Despachos A/B/C/D	Caracteres A/B/C/D	CPU % A/B/C/D
SCHED_FIFO (1,0009)	2 / 1 / 1 / 1	5.000.000 / 0 / 0 / 0	99,95 / 0,00 / 0,01 / 0,04
SCHED_RR (1,9094)	10 / 9 / 9 / 8	1.206.817 / 1.777.829 / 1.234.617 / 780.737	20,98 / 31,39 / 26,47 / 21,17
SCHED_IDLE (1,9726)	211 / 213 / 213 / 210	1.254.514 / 1.258.704 / 1.169.865 / 1.316.917	26,01 / 24,67 / 23,12 / 26,20
SCHED_LOW_IDLE (2,4064)	250 / 241 / 237 / 240	1.250.070 / 1.309.042 / 1.288.497 / 1.152.391	24,63 / 26,24 / 25,59 / 23,54
Misto: LOW_IDLE/IDLE (1,1545)	182 / 184	1.700.693 / 3.299.307	29,88 / 70,12

Tabela 1. CPU é o tempo somado das threads; percentuais são relativos a essa soma. Os despachos do programa são estimados; confira-os nos traces.

Comparação suplementar com 2 CPUs (quatro threads): FIFO, 10,642 s de CPU total e 25,47/26,74/24,38/23,41%; RR, 10,271 s e 24,20/24,69/26,18/24,93%. O kernel registrou RT throttling activated no FIFO.

4. Análise e justificativa

SCHED_FIFO. A recebeu 99,95% do tempo de CPU e escreveu o buffer inteiro; B, C e D não escreveram. Com a mesma prioridade, A não bloqueia no laço e segue executando até terminar. O resultado ilustra o FIFO entre iguais e o risco de uma tarefa RT monopolizar a CPU.

SCHED_RR. As quatro threads usam prioridade 10; ao expirar a fatia de 100 ms, uma thread pronta cede lugar à próxima. Esse timeout é o fim do quantum, não um erro ou uma espera. Nesta rodada, houve 10/9/9/8 despachos estimados, mas a CPU variou de 20,98% a 31,39%. Despachos contam entradas, não duração; a contenção e os bloqueios no semáforo afetam os tempos. O trace permite conferir o rodízio real.

SCHED_IDLE e SCHED_LOW_IDLE. Em cada execução homogênea, todas as threads têm o mesmo peso e dividiram a CPU aproximadamente por igual. Para o misto, os pesos 3:1 (IDLE:LOW_IDLE) sugerem 75/25; medimos 70,12/29,88. A diferença é plausível pela contenção no semáforo e pelo término ao preencher o buffer. Ambas são políticas de trabalho de fundo; LOW_IDLE tem peso menor.

Projeto, usos e limites. O semáforo evita escritas concorrentes, mas serializa a seção crítica. FIFO pode atender uma tarefa RT que não deve ser interrompida por iguais; RR reparte a CPU entre tarefas RT da mesma prioridade. IDLE/LOW_IDLE servem a manutenção em segundo plano e cedem CPU às classes superiores. Deadline não foi avaliada. Os dados são uma rodada por caso, sem médias.

5. Traces no KernelShark — inserir os prints

Insira um print de cada trace, identificando a política e as threads.

INSERIR PRINT: SCHED_FIFO	INSERIR PRINT: SCHED_RR
INSERIR PRINT: SCHED_IDLE	INSERIR PRINT: SCHED_LOW_IDLE
INSERIR PRINT: MISTO: LOW_IDLE + IDLE	INSERIR PRINT: área livre